

Introdução

O presente relatório descreve o desenvolvimento de uma série de missões práticas da disciplina de Computação Gráfica, utilizando a linguagem Python juntamente com a biblioteca OpenGL/GLUT. O objetivo principal das atividades foi aplicar conceitos fundamentais de modelagem geométrica 2D, renderização gráfica, interação por teclado, manipulação de primitivas gráficas e organização orientada a objetos.

Durante o desenvolvimento das missões, foi adotada uma arquitetura baseada em Programação Orientada a Objetos (POO), utilizando estruturas de vértices compartilhados (Shared Vertex), triangulação de objetos e renderização através de primitivas do OpenGL, como `GL_TRIANGLES` e `GL_LINES`.

Além disso, a resolução dessa atividade permitiu explorar alguns conceitos importantes da Computação Gráfica, como: Sistemas de coordenadas; Modelagem geométrica; Interatividade etc.

Todas as missões foram desenvolvidas utilizando Python no ambiente Replit, juntamente com as bibliotecas `OpenGL.GL`, `OpenGL.GLUT` e `OpenGL.GLU`.

Missão 01

Objetivo

A primeira missão consistiu na criação de um retângulo cuja largura e altura são fornecidas pelo usuário. Além disso, ao pressionar a barra de espaço, tanto o retângulo quanto o plano de fundo deveriam receber novas cores aleatórias.

Desenvolvimento

Para a implementação da missão, foi criada uma classe chamada Rectangle, responsável por armazenar: largura; altura; cor; vértices do objeto. O retângulo foi construído utilizando Shared Vertex, armazenando os quatro vértices:

```
# ===== Classe do Retângulo ===== #
class Rectangle:

    def __init__(self, width, height):

        self.width = width
        self.height = height

        self.color = self.random_color()

    # ===== Shared Vertex ===== #
    self.vertices = [
        (-width / 2, height / 2), # v0
        ( width / 2, height / 2), # v1
        ( width / 2, -height / 2), # v2
        (-width / 2, -height / 2) # v3
    ]
```

A troca de cores foi implementada utilizando eventos de teclado do GLUT, especificamente detectando o clique da tecla espaço por parte do usuário, com a chamada de uma função específica (update_color).

```

# ===== Atualizar cor ===== #
def update_color(self):
    self.color = self.random_color()

# ===== Desenhar ===== #
def draw(self):

    glColor3f(*self.color)

    glBegin(GL_TRIANGLES)

    for index in self.indices:
        glVertex2f(*self.vertices[index])

    glEnd()

```

```

# ===== Teclado ===== #
def keyboard(key, x, y):

    global bg_color

    # Barra de espaço
    if key == b' ':

        rect.update_color()

        bg_color = (
            random.random(),
            random.random(),
            random.random()
        )

        glutPostRedisplay()

```

A entrada de dados por parte do usuário foi feito via terminal

```
Rodando main.py...  
Digite a largura do retângulo: 20  
Digite a altura do retângulo: 30
```



Missão 02

Objetivo

A segunda missão teve como objetivo criar uma cena contendo quatro formas geométricas diferentes: quadrado; triângulo; círculo; hexágono. Cada forma deveria ocupar uma região diferente da janela e possuir cores distintas.

Desenvolvimento

Foi criada uma classe base chamada Shape, responsável por armazenar propriedades comuns das formas geométricas, como: posição; cor; lista de vértices; lista de índices.

```

# ===== Classe Base ===== #
class Shape:

    def __init__(self, x, y, color):

        self.x = x
        self.y = y
        self.color = color

        self.vertices = []
        self.indices = []

    def draw(self):

        glColor3f(*self.color)

        glBegin(GL_TRIANGLES)

        for index in self.indices:
            vx, vy = self.vertices[index]

            glVertex2f(vx + self.x, vy + self.y)

        glEnd()

```

A partir dessa classe, foram implementadas subclasses específicas para cada forma geométrica. O círculo foi construído proceduralmente através de múltiplos vértices distribuídos ao redor de uma circunferência utilizando funções trigonométricas (cos e sin). Esses vértices foram triangulados utilizando um vértice central. O hexágono foi desenvolvido utilizando a mesma lógica geométrica radial.

```

# ===== Quadrado ===== #
class Square(Shape):

    def __init__(self, x, y, size, color):

        super().__init__(x, y, color)

        s = size / 2

        # Shared vertex
        self.vertices = [
            (-s, s), # v0
            (s, s), # v1
            (s, -s), # v2
            (-s, -s) # v3
        ]

        self.indices = [
            0, 1, 2,
            2, 3, 0
        ]

# ===== Triângulo ===== #
class Triangle(Shape):

    def __init__(self, x, y, size, color):

        super().__init__(x, y, color)

        s = size

        self.vertices = [
            (0, s),
            (-s, -s),
            (s, -s)
        ]

        self.indices = [
            0, 1, 2
        ]

```

```

# ===== Hexágono =====
class Hexagon(Shape):

    def __init__(self, x, y, radius, color):

        super().__init__(x, y, color)

        # Centro
        self.vertices.append((0, 0))

        # Vértices externos
        for i in range(6):

            angle = math.radians(i * 60)

            vx = radius * math.cos(angle)
            vy = radius * math.sin(angle)

            self.vertices.append((vx, vy))

        # Triângulos
        for i in range(1, 7):

            next_i = i + 1 if i < 6 else 1

            self.indices.extend([
                0, i, next_i
            ])

```

```

# ===== Circulo ===== #
class Circle(Shape):

    def __init__(self, x, y, radius, color, segments=30):

        super().__init__(x, y, color)

        # Centro
        self.vertices.append((0, 0))

        # Circunferência
        for i in range(segments):

            angle = 2 * math.pi * i / segments

            vx = radius * math.cos(angle)
            vy = radius * math.sin(angle)

            self.vertices.append((vx, vy))

        # Índices
        for i in range(1, segments):

            self.indices.extend([
                0, i, i + 1
            ])

        self.indices.extend([
            0, segments, 1
        ])

```

Cada objeto foi desenhado em uma região específica da tela, simulando um painel geométrico.

```
# ===== Formas ===== #

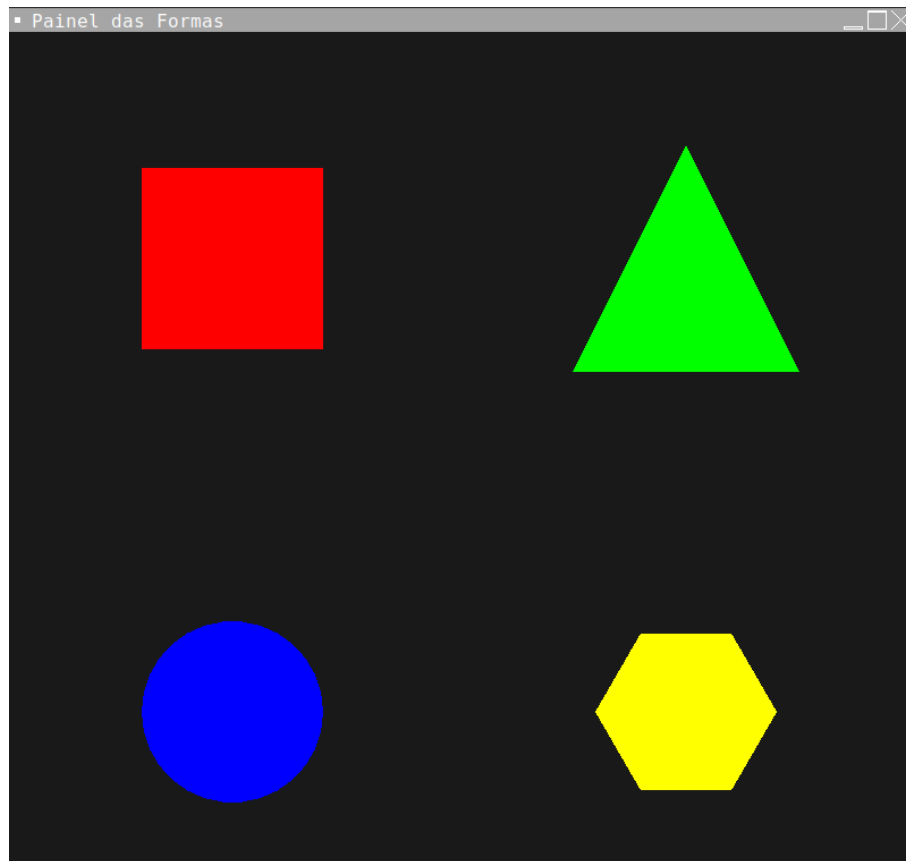
shapes = [

    Square(
        x=-50,
        y=50,
        size=40,
        color=(1, 0, 0)
    ),

    Triangle(
        x=50,
        y=50,
        size=25,
        color=(0, 1, 0)
    ),

    Circle(
        x=-50,
        y=-50,
        radius=20,
        color=(0, 0, 1)
    ),

    Hexagon(
        x=50,
        y=-50,
        radius=20,
        color=(1, 1, 0)
    )
]
```

Missão 03

Objetivo

A terceira missão consistiu na criação de uma função capaz de desenhar triângulos isósceles utilizando base e altura fornecidas pelo usuário.

Além disso, deveriam ser desenhados pelo menos cinco triângulos com:

- tamanhos diferentes;
- posições diferentes;
- cores distintas.

Desenvolvimento

Foi criada uma classe chamada IsoscelesTriangle, derivada da classe base Shape.

```
# ===== Triângulo Isósceles ===== #
class IsoscelesTriangle(Shape):

    def __init__(self, x, y, base, height, color):

        super().__init__(x, y, color)

        half_base = base / 2
        half_height = height / 2

        # Shared vertex
        self.vertices = [

            (0, half_height),           # topo

            (-half_base, -half_height), # esquerda

            (half_base, -half_height)    # direita

        ]

        # Índices
        self.indices = [

            0, 1, 2

        ]
```

Cada triângulo foi construído utilizando três vértices: vértice superior; vértice inferior esquerdo; vértice inferior direito. A posição dos vértices foi calculada utilizando os valores de base e altura fornecidos pelo usuário.

A cena foi composta por múltiplas instâncias da mesma classe, variando: escala; posição; cor. Essa abordagem permitiu a reutilização geométrica e organização eficiente da cena.

```
# ===== Criar Triângulo ===== #
def create_triangle():

    base = float(input("Digite a base do triângulo: "))
    height = float(input("Digite a altura do triângulo: "))

    return base, height

# ===== Programa Principal ===== #
def main():

    global triangles

    # Entrada do usuário
    base, height = create_triangle()

    glutInit()

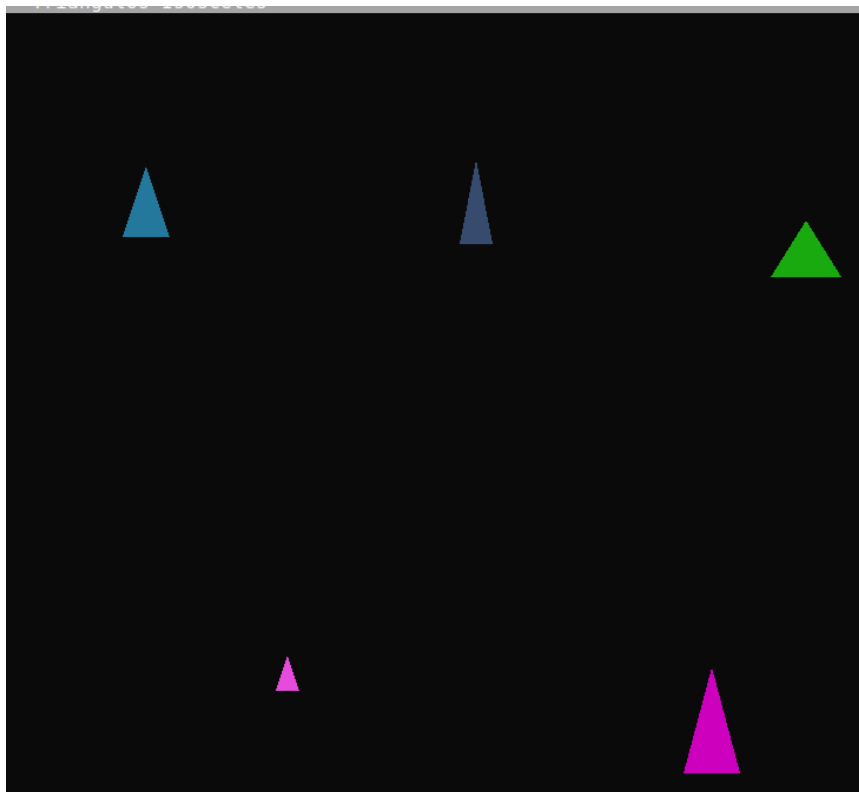
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB)

    glutInitWindowSize(800, 800)

    glutCreateWindow(b"Triangulos Isosceles")

    init()
```

```
# ===== Cena ===== #
triangles = [
    IsoscelesTriangle(
        x=-70,
        y=60,
        base=base,
        height=height,
        color=random_color()
    ),
    IsoscelesTriangle(
        x=0,
        y=60,
        base=base * 0.7,
        height=height * 1.2,
        color=random_color()
    ),
    IsoscelesTriangle(
        x=70,
        y=50,
        base=base * 1.5,
        height=height * 0.8,
        color=random_color()
    ),
    IsoscelesTriangle(
        x=-40,
        y=-40,
        base=base * 0.5,
        height=height * 0.5,
        color=random_color()
    ),
    IsoscelesTriangle(
        x=50,
        y=-50,
        base=base * 1.2,
        height=height * 1.5,
        color=random_color()
    )
]
```



```
Rodando missao03.py...
```

```
Digite a base do triângulo: 10
```

```
Digite a altura do triângulo: 15
```

Missão 04

Objetivo

A quarta missão teve como objetivo criar uma cena contendo triângulos distribuídos em diferentes quadrantes do plano cartesiano.

Além disso, deveriam ser desenhados:

- eixo X;
- eixo Y;
- coordenadas textuais de cada triângulo.

Desenvolvimento

Foi utilizada a função `gluOrtho2D()` para configurar o Sistema de Referência do Usuário (SRU), posicionando a origem no centro da tela. Os eixos foram desenhados utilizando a primitiva `GL_LINES`.

```
# ===== Inicializaçã
def init():

    glClearColor(0.08, 0.08, 0.08, 1)

    glMatrixMode(GL_PROJECTION)

    glLoadIdentity()

    gluOrtho2D(-100, 100, -100, 100)
```

```
# ===== Desenhar Eixos =====
def draw_axes():

    glLineWidth(2)

    glColor3f(1, 1, 1)

    glBegin(GL_LINES)

    # Eixo X
    glVertex2f(-100, 0)
    glVertex2f(100, 0)

    # Eixo Y
    glVertex2f(0, -100)
    glVertex2f(0, 100)

    glEnd()
```

Os triângulos foram posicionados estrategicamente nos quadrantes I, II, III e IV por meio de

```
# ===== Triângulos ===== #

triangles = [

    # Quadrante I
    Triangle(
        x=50,
        y=50,
        base=20,
        height=30,
        color=(1, 0, 0)
    ),
    # Quadrante II
    Triangle(
        x=-50,
        y=40,
        base=25,
        height=35,
        color=(0, 1, 0)
    )
]
```

```
# Quadrante III
Triangle(
    x=-40,
    y=-50,
    base=30,
    height=25,
    color=(0, 0, 1)
),
# Quadrante IV
Triangle(
    x=50,
    y=-20,
    base=30,
    height=25,
    color=(1, 0, 1)
)
]
```

As coordenadas dos objetos foram exibidas utilizando a função `glutBitmapCharacter()`, responsável pela renderização de texto em OpenGL clássico.

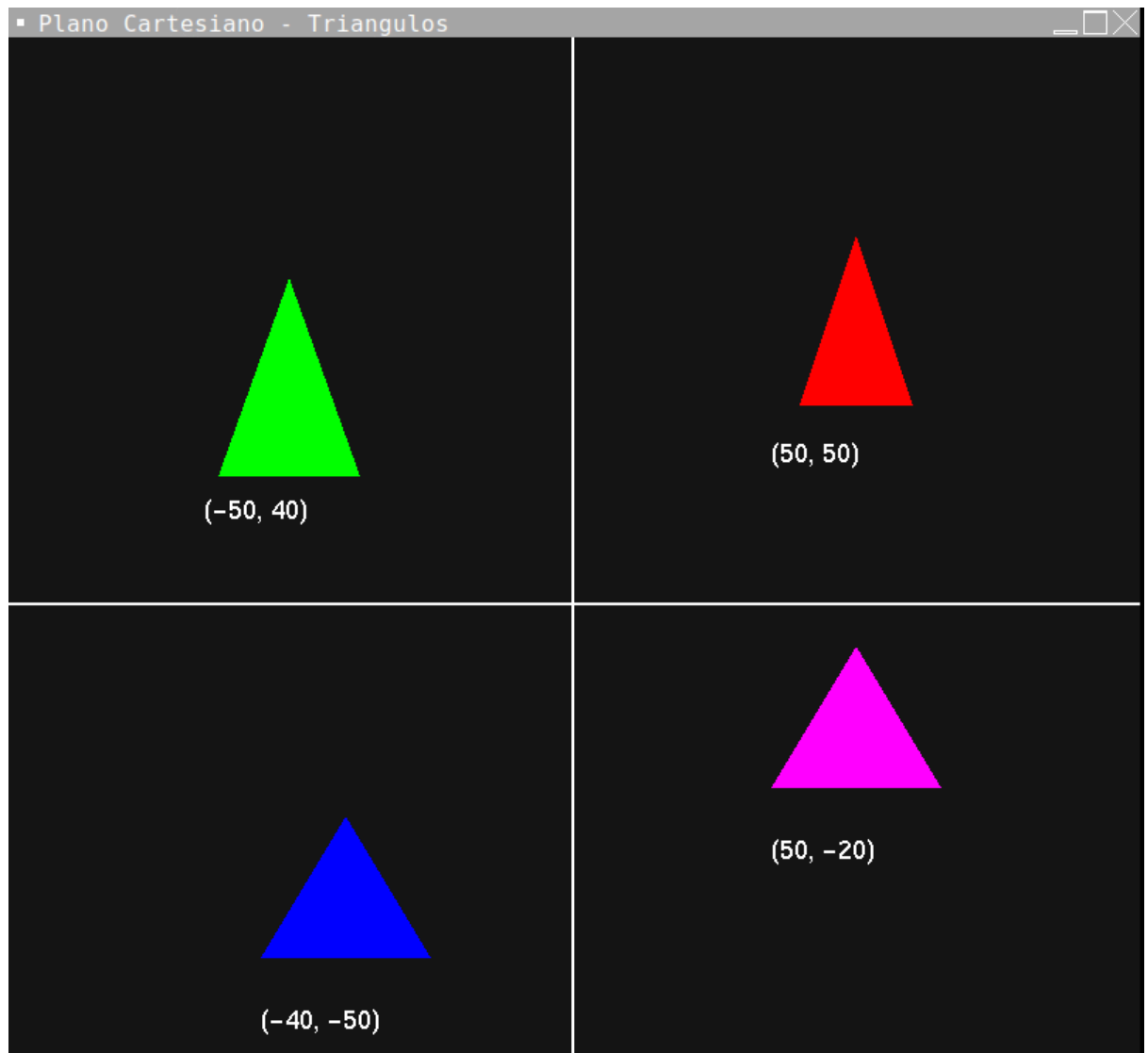
```
# ===== Desenhar Texto
def draw_text(text, x, y):

    glColor3f(1, 1, 1)

    glRasterPos2f(x, y)

    for char in text:

        glutBitmapCharacter(
            GLUT_BITMAP_HELVETICA_18,
            ord(char)
        )
```



Missão 05

Objetivo

A quinta missão consistiu na criação de uma representação geométrica da Insígnia de Malta. Além disso, ao pressionar a tecla "C", as cores do símbolo deveriam ser alteradas aleatoriamente.

Desenvolvimento

Inicialmente, a insígnia foi pensada utilizando retângulos e triângulos. Entretanto, posteriormente foi desenvolvida uma solução procedural mais sofisticada utilizando:

- trigonometria;
- coordenadas polares;
- construção radial.

A figura foi construída através de quatro braços simétricos distribuídos em torno da origem.

Cada braço foi formado por dois triângulos conectados, gerando o formato característico da Cruz de Malta.

As posições dos vértices foram calculadas matematicamente utilizando:

- seno ($\sin$);
- cosseno ($\cos$);
- ângulos distribuídos radialmente.

```

# ===== Triângulo ===== #
class Triangle(Shape):
    def __init__(self, vertices, color):
        super().__init__(vertices, color)
        self.indices = [0, 1, 2]

# ===== Insignia de Malta ===== #
class MaltaInsignia:
    def __init__(self):
        self.triangles = []
        self.create_insignia()

# ===== Construção ===== #
    def create_insignia(self):
        outer_radius = 90 # distância do centro às 8 pontas externas
        notch_radius = 60 # distância do centro ao entalhe em V de cada braço
        half_angle = 30 # meia largura angular de cada braço (graus)

        color = self.random_color()
        center = (0.0, 0.0)

        # 4 braços: cima, direita, baixo, esquerda
        arm_directions = [90, 0, -90, 180]

        for arm_dir in arm_directions:
            a_left = math.radians(arm_dir + half_angle)
            a_right = math.radians(arm_dir - half_angle)
            a_mid = math.radians(arm_dir)

            tip_left = (outer_radius * math.cos(a_left), outer_radius * math.sin(a_left))
            tip_right = (outer_radius * math.cos(a_right), outer_radius * math.sin(a_right))
            notch = (notch_radius * math.cos(a_mid), notch_radius * math.sin(a_mid))

            # Dois triângulos por braço formando o V externo
            self.triangles.append(Shape([center, tip_left, notch], color))
            self.triangles.append(Shape([center, notch, tip_right], color))

```

A interação foi implementada através do evento de teclado do GLUT, permitindo alterar dinamicamente as cores do símbolo.

```

# ===== Cor Aleatória =====
def random_color(self):

    return (
        random.random(),
        random.random(),
        random.random()
    )

# ===== Desenhar =====
def draw(self):
    for tri in self.triangles:
        tri.draw()

# ===== Atualizar cores =====
def energize(self):
    for tri in self.triangles:
        tri.randomize_color()

# ===== Objeto Global =====
insignia = MaltaInsignia()

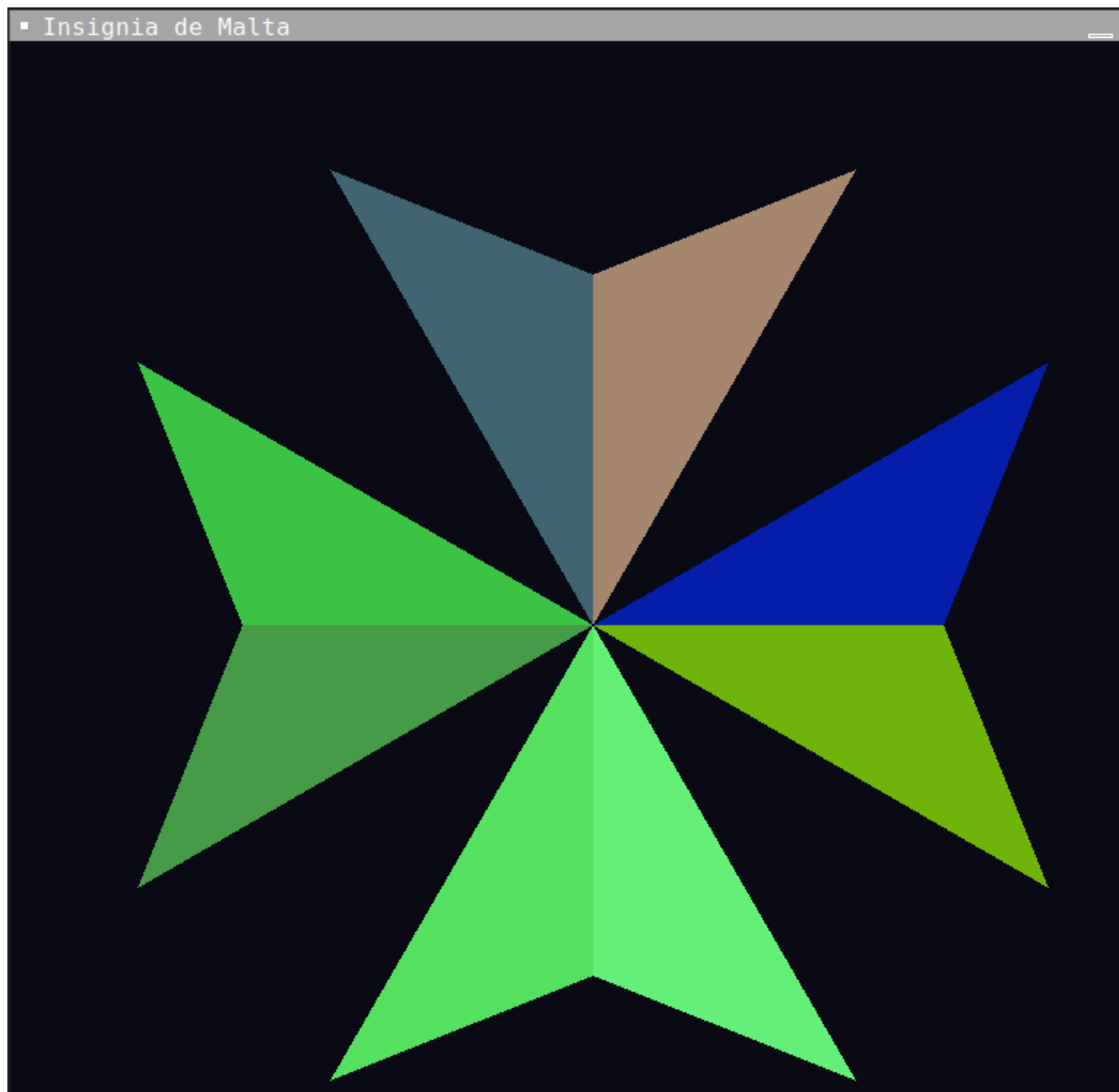
# ===== Display =====
def display():
    glClear(GL_COLOR_BUFFER_BIT)
    insignia.draw()
    glutSwapBuffers()

# ===== Teclado =====
def keyboard(key, x, y):
    # Pressione C
    if key == b'c' or key == b'C':
        insignia.energize()
        glutPostRedisplay()

```

▪ Insignia de Malta





Missão 06

Objetivo

A sexta missão teve como objetivo construir uma constelação formada por múltiplas estrelas representadas por círculos de diferentes tamanhos.

As estrelas deveriam:

- possuir cores aleatórias;
- estar conectadas por linhas;
- permitir interação dinâmica através do teclado.

Os recursos interativos incluíam:

- adicionar novas estrelas;
- remover estrelas;
- reiniciar a constelação;
- ativar modo noturno.

Desenvolvimento

Foi criada uma classe chamada Star, responsável pela construção procedural dos círculos.

Cada círculo foi construído utilizando:

- um vértice central;
- múltiplos vértices distribuídos radialmente;

```

# ===== CLASSE ESTRELA =====
class Star:
    def __init__(self, x, y, radius, color):

        self.x = x
        self.y = y

        self.radius = radius

        self.color = color

        self.vertices = []
        self.indices = []

        self.generate_circle()

# ===== GERAÇÃO DO CÍRCULO ===== #

    def generate_circle(self, segments=40):

        # vértice central
        self.vertices.append((0, 0))

        # borda do círculo
        for i in range(segments):

            angle = 2 * math.pi * i / segments

            vx = self.radius * math.cos(angle)
            vy = self.radius * math.sin(angle)

            self.vertices.append((vx, vy))

```

Além disso, foi criada uma classe chamada Constellation, responsável pelo gerenciamento dinâmico da cena.

Essa classe controlava:

- criação de estrelas;
- remoção;
- atualização de cores;
- modo noturno;
- reinicialização da constelação.

```
# ===== CONSTELAÇÃO =====
✓ class Constellation:
✓     def __init__(self):
        self.stars = []
        self.night_mode = False
        self.generate_initial_stars()

        # COR ALEATÓRIA
✓     def random_color(self):
✓         return (
            random.random(),
            random.random(),
            random.random()
        )

        # MODO NOTURNA
✓     def night_color(self):
✓         colors = [
            (1, 1, 1),          # branco
            (1, 1, 0.6),        # amarelo claro
            (1, 0.9, 0.5)
        ]

        return random.choice(colors)
```



```
# ===== CRIAÇÃO DAS ESTRELAS ===== #
def create_star(self):

    x = random.randint(MIN_COORD, MAX_COORD)

    y = random.randint(MIN_COORD, MAX_COORD)

    radius = random.randint(MIN_RADIUS, MAX_RADIUS)

    if self.night_mode:
        color = self.night_color()
    else:
        color = self.random_color()

    return Star(x, y, radius, color)

# ===== GERAÇÃO INICIAL ===== #

def generate_initial_stars(self):

    self.stars.clear()

    for _ in range(INITIAL_STARS):

        self.stars.append(
            self.create_star()
        )
```

```

# ADICIONAR ESTRELA
def add_star(self):
    self.stars.append(
        self.create_star()
    )

# REMOVER ESTRELA
def remove_star(self):
    if len(self.stars) > 0:
        index = random.randint(
            0,
            len(self.stars) - 1
        )
        self.stars.pop(index)

# RESETAR
def reset(self):
    self.generate_initial_stars()

# MODO NOTURNO
def toggle_night_mode(self):
    self.night_mode = not self.night_mode

    for star in self.stars:
        if self.night_mode:
            star.color = self.night_color()
        else:
            star.color = self.random_color()

```

As conexões entre estrelas foram desenhadas utilizando GL_LINES.

```

# CONEXÕES
def draw_connections(self):

    glLineWidth(1.5)

    if self.night_mode:
        glColor3f(0.6, 0.6, 1)
    else:
        glColor3f(1, 1, 1)

    glBegin(GL_LINES)

    for i in range(len(self.stars) - 1):

        s1 = self.stars[i]
        s2 = self.stars[i + 1]

        glVertex2f(s1.x, s1.y)
        glVertex2f(s2.x, s2.y)

    glEnd()

# DESENHAR CONSTELAÇÃO
def draw(self):
    self.draw_connections()
    for star in self.stars:
        star.draw()

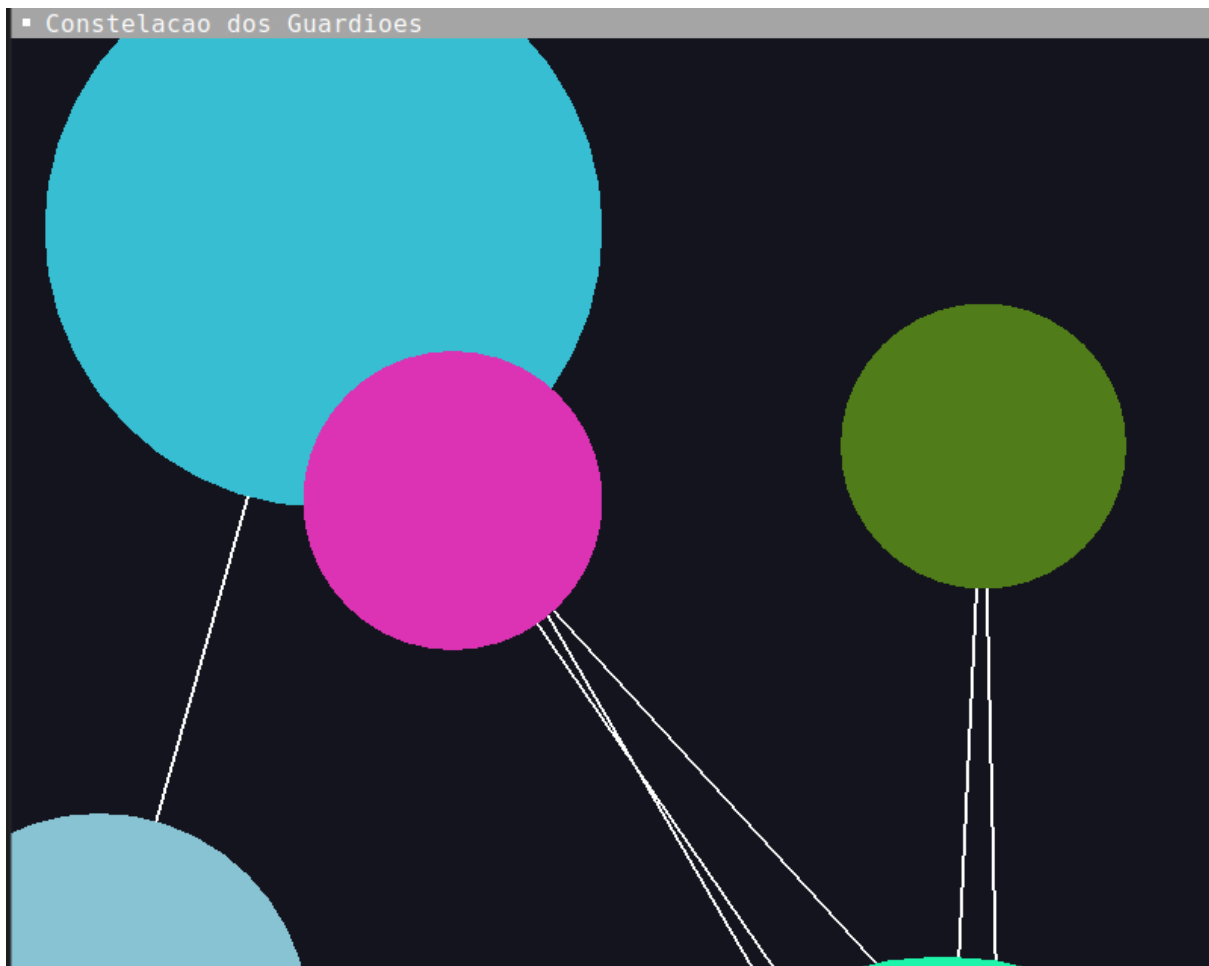
```

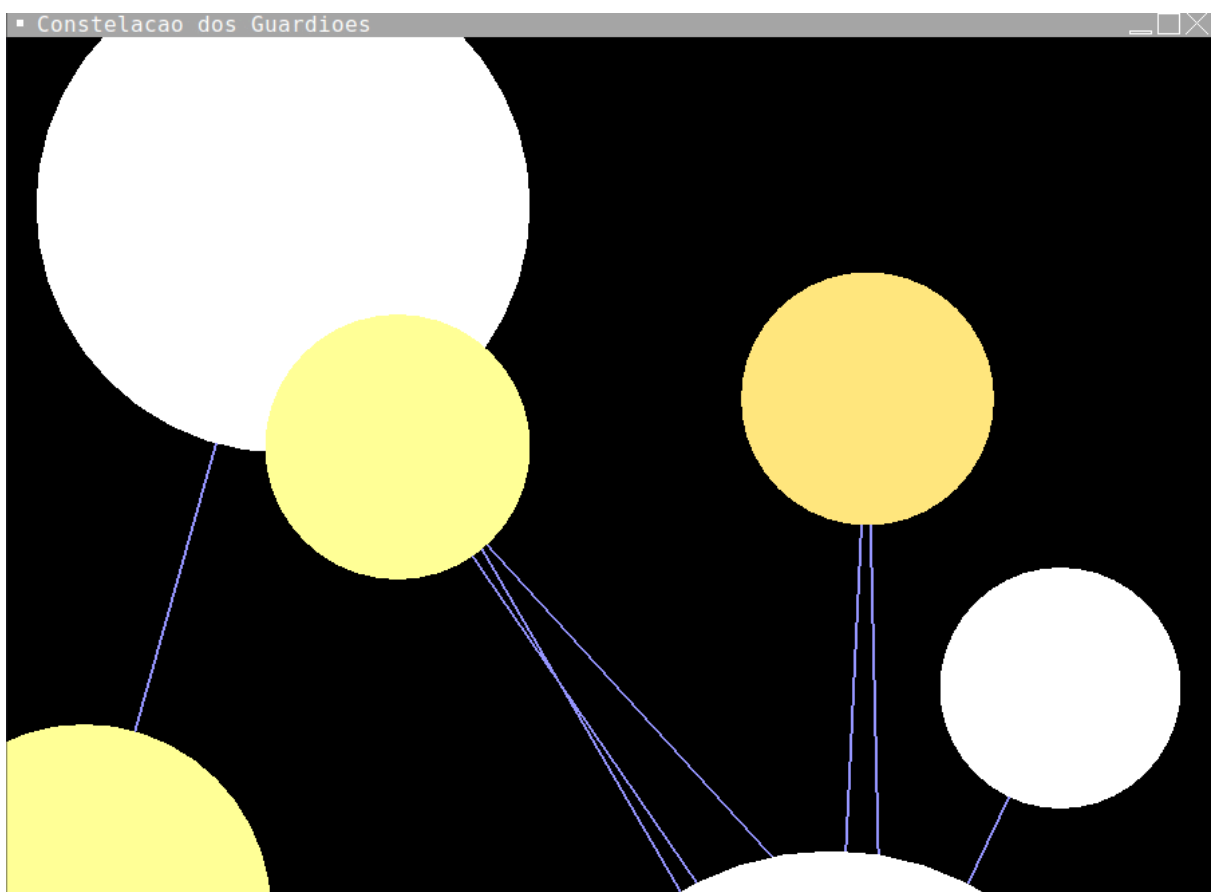
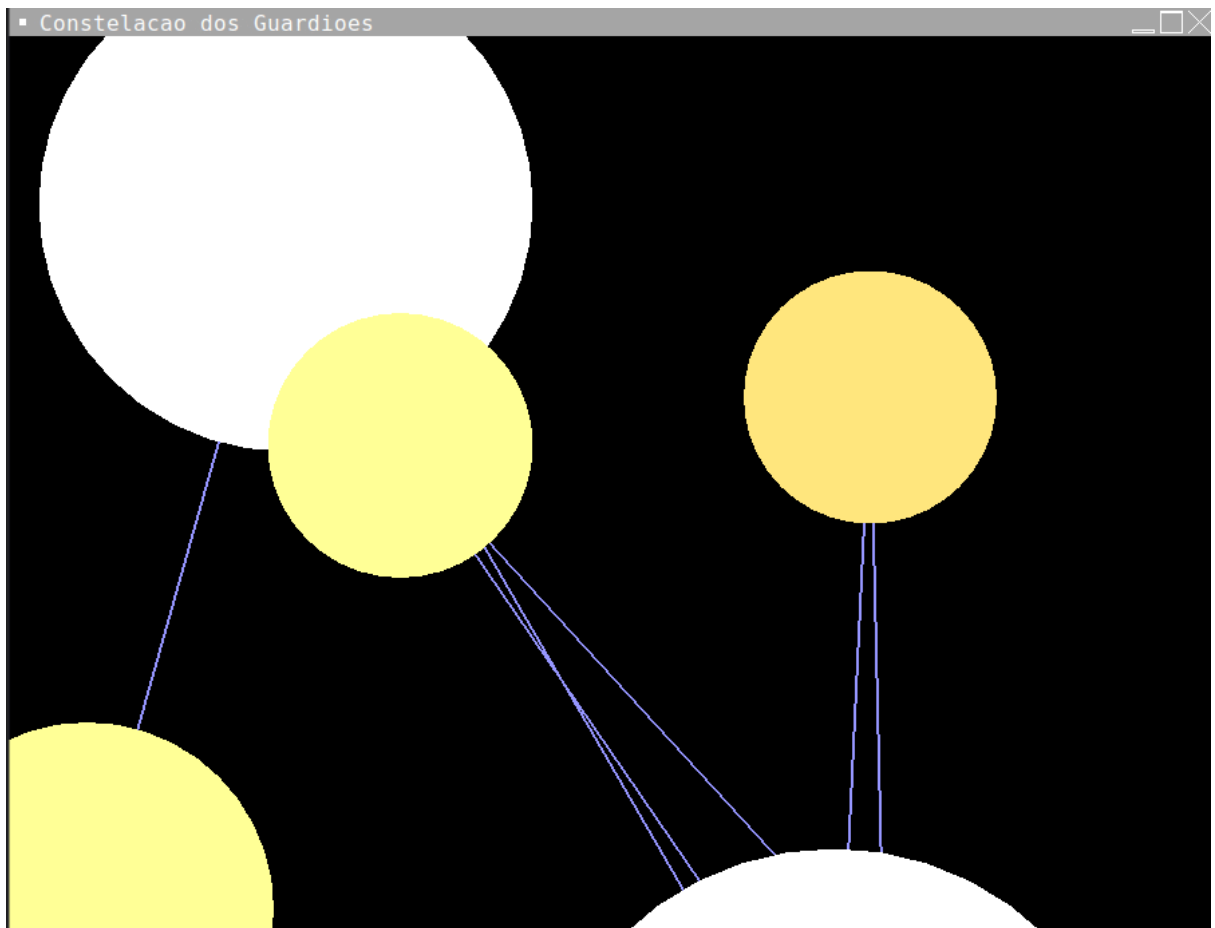
As interações foram implementadas através de eventos de teclado utilizando GLUT.

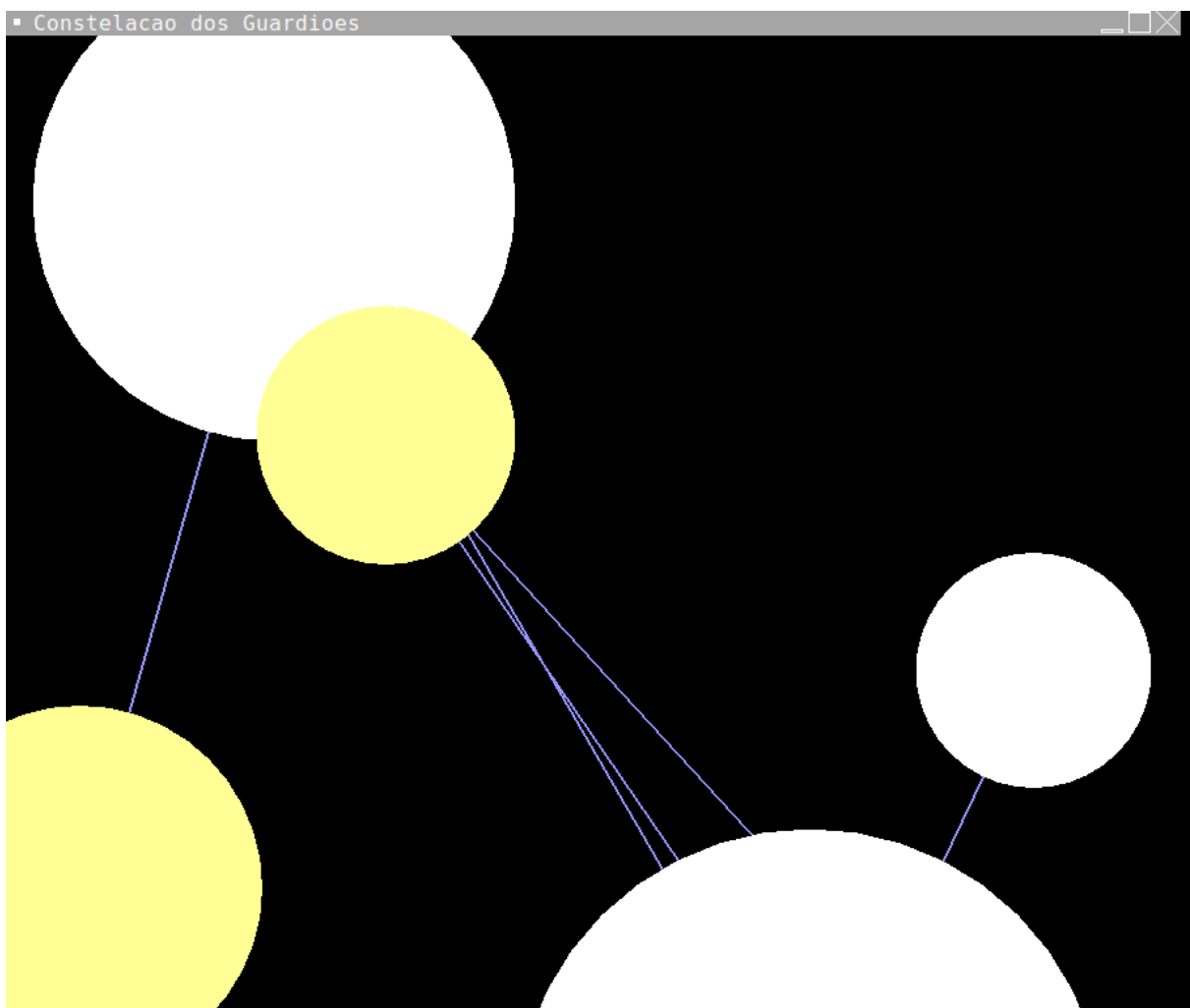
```

# ===== TECLADO ===== #
def keyboard(key, x, y):
    # adicionar estrela
    if key == b'n':
        constellation.add_star()
    # remover estrela
    elif key == b'x':
        constellation.remove_star()
    # resetar
    elif key == b'r':
        constellation.reset()
    # modo noturno
    elif key == b't':
        constellation.toggle_night_mode()
    glutPostRedisplay()

```







Considerações Finais

O desenvolvimento das seis missões permitiu aplicar diversos conceitos fundamentais da Computação Gráfica 2D utilizando OpenGL.

Ao longo das atividades, foi possível compreender melhor:

- modelagem geométrica;
- renderização baseada em vértices;
- triangulação;
- sistemas de coordenadas;
- interação gráfica;
- composição de objetos;
- geometria procedural.

A utilização de Programação Orientada a Objetos tornou o código mais organizado, reutilizável e modular, facilitando a expansão das funcionalidades ao longo das missões.

Além disso, o uso de Shared Vertex contribuiu para uma representação geométrica mais eficiente, reduzindo redundâncias e aproximando as implementações das técnicas utilizadas em aplicações gráficas reais.

As atividades também permitiram explorar recursos interativos, tornando as cenas mais dinâmicas e aproximando o projeto de conceitos utilizados em jogos, engines gráficas e sistemas de visualização computacional.

Conclusão

A realização das missões propostas possibilitou consolidar conhecimentos importantes da disciplina de Computação Gráfica, especialmente relacionados à construção de objetos bidimensionais utilizando OpenGL.

Por meio da implementação de diferentes cenas e sistemas interativos, foi possível compreender como primitivas geométricas simples podem ser combinadas para formar estruturas mais complexas.

Também foi possível observar a importância da organização do código utilizando Programação Orientada a Objetos e estruturas de vértices compartilhados, aproximando o projeto de práticas adotadas no desenvolvimento gráfico moderno.

Dessa forma, as atividades contribuíram significativamente para o desenvolvimento das habilidades de modelagem geométrica, lógica espacial, manipulação gráfica e programação interativa.

Anexos

Link dos códigos das questões no GitHub:

https://github.com/LucasSouza67/Programacao_com_objetos_2d

Referências

PYTHON SOFTWARE FOUNDATION. Python. Disponível em: <https://www.python.org/>. Acesso em: 8 maio 2026.

GOOGLE. Google Colaboratory. Disponível em:

<https://colab.research.google.com/notebooks/welcome.ipynb?hl=pt-BR>. Acesso em: 8 maio 2026.

OPENGL. OpenGL Documentation. Disponível em: <https://www.opengl.org/>. Acesso em: 8 maio 2026.

Aulas e materiais disponibilizados pelo professor da disciplina de Computação Gráfica.